UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Corso di Laurea Triennale in Informatica

Prototipo di App Web per la Visualizzazione di Mappe Standard OGC

Progetto di Tirocinio

Relatori:

Andrea Canciani
Alessio Conte

Candidato:

Samuele Calugi

ANNO ACCADEMICO 2023/2024

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di tirocinio formativo, della durata di circa trecento ore complessive, dal laureando Samuele Calugi presso l'azienda Geckosoft, sede di Pisa. Durante il tirocinio era richiesto il raggiungimento di massima dei seguenti obiettivi:

- studio autonomo degli Standard OGC (Web Map Service, Web Feature Service, Web Coverage Service, Web Map Tile Service) e dei formati dati geospaziali (GeoJSON, Shapefile, ecc.);
- sviluppo di un frontend interattivo in grado di mostrare mappe ottenute attraverso gli standard OGC e i formati dati geospaziali specificati;
- sviluppo di un backend per il fornimento delle mappe in formato conforme agli standard OGC e ai formati dati geospaziali richiesti;
- integrazione del backend e del frontend per creare un'applicazione completa e funzionale per la visualizzazione di dati geospaziali provenienti da diverse fonti.
- l'utilizzo di *Angular*, per la parte front-end e *.NET* per la parte back-end;
- documentazione dettagliata del lavoro svolto, incluse le tecnologie utilizzate, le scelte progettuali e le modalità di utilizzo del prodotto.

Indice

1 Introduzione **9**
1.1 Introduzione al progetto 9
1.2 Base di partenza 10
1.3 Contesto di Sviluppo 10
1.4 Struttura della Relazione 11

2 Tecnologie di interesse **13**
2.1 Gli standard forniti da OGC 13
2.1.1 Protocollo WMS 14
2.1.2 Protocollo WMTS 17
2.1.3 Protocollo WFS 19
2.1.4 Protocollo WCS 23
2.2 Tiled Map Service (TMS) 24
2.3 Formati di file GeoSpaziali 27
2.4 OpenLayers 28
2.5 GeoServer 29

3 Introduzione al Tirocinio **33**
3.0.1 Introduzione al Problema 33
3.0.2 Cosa è stato Realizzato 33

4 Tecnologie Aziendali Utilizzate **35**
4.0.1 Microservizi e Docker 35
4.0.2 WebServer e Reverse Proxy 36
4.0.3 Front-end e Back-end 36
4.0.4 Continuous Integration e Delivery 37

5 Svolgimento del Tirocinio **39**
5.1 Preparazione al Tirocinio 39
5.1.1 Studio di Reactive Programming e Angular 39
5.1.2 Realizzazione di un prototipo con OpenLayers 40
5.1.3 Studio di OpenLayers in Angular 41
5.1.4 Censimento delle Mappe 42
5.2 Sviluppo Front-end con OpenLayers 43

5.2.1 Perché OpenLayers 43
5.2.2 Implementazione nel Progetto 45

Elenco delle figure

2.1 Diagramma di Sequenza del protocollo WMS 15
2.2 Diagramma di Sequenza del protocollo WMTS 18
2.3 Diagramma di Sequenza del protocollo WFS 21
2.4 Tiles con livello di zoom 25

Capitolo 1

Introduzione

Il presente capitolo fornisce una panoramica del progetto di tirocinio, durato trecento ore, offrendo una spiegazione sull'ambito del progetto e indicando le tecnologie impiegate per la sua realizzazione.

1.1 Introduzione al progetto

Il progetto di tirocinio mira allo sviluppo e l'integrazione di un prototipo di applicazione per la visualizzazione di mappe geospaziali, avvalendosi degli standard dell'*Open Geospatial Consortium* (*OGC*) e delle più moderne tecnologie web.
L'iniziativa nasce dalla crescente importanza dei dati geospaziali nelle applicazioni web, e della necessità di fornire strumenti interattivi, e intuitivi, per la loro visualizzazione e manipolazione.
Il lavoro svolto, della durata di circa trecento ore, si concentra sia sull'implementazione di un'applicazione front-end per la visualizzazione di mappe geospaziali, includendo un'interfaccia utente interattiva, sia sullo sviluppo di un back-end per la gestione e il fornimento dei dati delle mappe.
Lo sviluppo si è concentrato su un'applicazione già esistente, denominata *Inspicio*, creata dall'azienda Geckosoft in collaborazione con le facoltà di Informatica e di Ingegneria Informatica dell'Università di Pisa.
Tale applicazione nasce con l'obiettivo di creare una piattaforma completa per il censimento digitale e la valutazione del rischio dei ponti in tutta Italia [1]. La piattaforma offre agli utenti la possibilità di raccogliere dati dettagliati sui ponti, compresi parametri come posizione geografica, dimensioni, materiali di costruzione, età e condizioni attuali. Attraverso una mappa interattiva dell'Italia, è possibile esplorare dettagliatamente tali dati. Talvolta però ci sono delle informazioni, relative a un ponte, che non dipendono tanto dal ponte stesso quanto dalla zona ove è eretto. Tra queste troviamo: possibile attività sismiche, pericolo alluvioni, esondazione di fiumi,

[1] Con censimento digitale di un ponte si intende la raccolta, organizzazione e manutenzione di tutti i dati relativi a tale ponte, con lo scopo di tenerne sotto controllo lo stato in maniera continuativa.

frane, etc.. Per tale motivo, vengono implementate delle mappe che descrivono questi fenomeni, e che possono essere sovrapposte per ottenere un quadro più generale del ponte considerato.

1.2 Base di partenza

All'inizio del tirocinio, l'architettura del software e lo stack di sviluppo per la piattaforma Inspicio erano già stati selezionati. Tuttavia, la componente che risultava totalmente assente era quella relativa alle mappe, senza né front-end né back-end definiti. Per questo motivo, il candidato, assieme al tutore aziendale, si sono occupati di valutare e scegliere le tecnologie che meglio avrebbero consentito la gestione di mappe geospaziali. Dopodiché il candidato è stato incaricato dello sviluppo di questo aspetto. La precedente mancanza delle mappe si riconduce ad altre priorità degli sviluppatori già presenti in azienda.

1.3 Contesto di Sviluppo

La piattaforma, che prende l'acronimo di *BMS* (*Bridge Management System*) è stata sviluppata come un'applicazione Web moderna, seguendo un'architettura a microservizi che favorisce la modularità e la scalabilità del sistema. Questa scelta architetturale è stata preferita rispetto alle tradizionali applicazioni monolitiche, in quanto ha permesso di separare i vari servizi, semplificandone la gestione e consentendo una maggiore flessibilità nell'implementazione di nuove funzionalità. Ciò ha portato all'adozione di un ambiente di sviluppo basato su un approccio containerizzato utilizzando *Docker*. L'utilizzo di container Docker ha semplificato notevolmente la gestione delle dipendenze e delle configurazioni dei servizi, garantendo al contempo una maggiore portabilità dell'applicazione tra diversi ambienti di sviluppo e produzione.
Per quanto concerne le tecnologie impiegate nello sviluppo dell'interfaccia grafica della piattaforma, è stato selezionato *Angular*, un framework JavaScript moderno e robusto, caratterizzato da una struttura modulare che offre una vasta gamma di librerie. Queste caratteristiche lo rendono particolarmente adatto per la creazione di interfacce utente dinamiche e complesse, come nel caso della piattaforma Inspicio.
Per il back-end, è stata selezionata la piattaforma .NET, un framework di sviluppo completo e versatile supportato da Microsoft. .NET offre una vasta gamma di strumenti e librerie che semplificano lo sviluppo, la manutenzione e la scalabilità delle applicazioni.
L'adozione di Angular e .NET come principali framework di sviluppo hanno consentito agli sviluppatori di seguire un pattern unificato attraverso una varietà di progetti. Questa scelta ha ridotto la confusione e garantito una maggiore coerenza nelle implementazioni, assicurando che tutti i software sviluppati seguissero gli stessi standard e convenzioni. Si tratta di un aspetto cruciale in contesti aziendali con

molti progetti in corso, facilitando la gestione e la manutenzione dell'intero insieme di software.
Infine, nella comunicazione tra front-end e back-end della piattaforma, è stato adottato il paradigma delle *Rest API*, che rappresenta uno standard per lo scambio di dati tra client e server su protocollo HTTP. Più specificamente, le API sono esposte tramite Swagger UI, consentendo al front-end di interagire con esse in modo intuitivo e documentato. Il protocollo utilizzato per definire le specifiche delle API rispecchia lo standard di OpenAPI 3 (OAS3), garantendo una chiara e precisa descrizione delle operazioni e dei dati scambiati tra front-end e back-end.

1.4 Struttura della Relazione

Il secondo capitolo descrive le tecnologie studiate e utilizzate per lo sviluppo del progetto.

Il terzo capitolo approfondisce il percorso di tirocinio formativo.

Capitolo 2

Tecnologie di interesse

Questo capitolo si occupa di spiegare le tecnologie che sono state impiegate durante il tirocinio, al fine di comprendere successivamente le scelte progettuali intraprese durante lo sviluppo.
Particolare attenzione sarà dedicata agli standard definiti dall'Open Geospatial Consortium (OGC), tra cui Web Map Service (WMS), il Web Map Tiles Service (WMTS), Web Feature Service (WFS) e Web Coverage Service (WCS) e dall'Open Source Geospatial Foundation (OSGEO) tramite il protocollo Tile Map Service (TMS).
Si tratta di protocolli universalmente riconosciuti per la distribuzione e l'accesso ai dati geospaziali tramite Internet. Saranno inoltre analizzate altre tecnologie rilevanti per la gestione e la visualizzazione di dati geospaziali (come GeoJSON e ShapeFile), formati comuni per lo scambio di dati geografici. Infine verranno esplicate OpenLayers e GeoServer, due tecnologie fondamentali per la visualizzazione e la gestione di dati geospaziali.

2.1 Gli standard forniti da OGC

Il consorzio Open Geospatial Consortium (OGC) ha definito diversi standard per i servizi web che consentono ai client di accedere e interrogare informazioni geografiche. Tra questi troviamo:

- Web Map Service (WMS);
- Web Map Tile Service (WMTS);
- Web Feature Service (WFS);
- Web Coverage Service (WCS).

Come spiegato nella documentazione [13], ciascuno di questi standard offre funzionalità specifiche per soddisfare diverse esigenze di accesso ai dati geospaziali. Questi servizi si basano su altre tecnologie come l'architettura REST e il protocollo SOAP, che trasmettono i messaggi di risposta ai client in formato XML.

Lo standard OGC prevede che tutti i client che necessitano di contattare il servizio debbano prima effettuare una richiesta HTTP denominata *GetCapabilities*. Questa operazione è necessaria, per il client, per poter sapere quali informazioni possono essere reperite da quel servizio. In generale, queste informazioni comprendono: le mappe (una o più), le proiezioni che supportano, i loro formati (PNG, JPG, GML) e altre richieste HTTP che si possono fare al servizio stesso.

2.1.1 Protocollo WMS

Il Web Map Service (WMS) è universalmente riconosciuto come uno dei principali standard nel contesto dei servizi geospaziali, ed è tra i più importanti per la distribuzione di dati di mappe attraverso la rete Internet. Questo protocollo si occupa di offrire una rappresentazione visiva delle mappe geografiche distribuendole, essenzialmente sotto forma di immagini raster, come PNG o JPG, ma possono anche essere trasmesse in un formato vettoriale come SVB o WebCGM.
Il WMS, in maniera analoga agli altri servizi, offre un'interfaccia basata su protocollo HTTP che consente ai client di richiedere informazioni sulle mappe, sotto forma di richiesta XML. Nello specifico, un client che vuole contattare un servizio WMS, esegue per prima la richiesta GetCapabilities. Le GetCapabilities del servizio WMS contengono: le proiezioni supportate dalle mappe, i formati che il server può restituire e tutte le mappe che possiede (chiamate *layer*). Una volta ricevute queste informazioni, il client, invia la richiesta GetMap, che vuole come argomenti la proiezione, il layer che vuole visualizzare e il formato dell'immagine che vuole ricevere. Dunque, il server WMS restituisce l'intera immagine della mappa.
Inoltre, lo standard WMS supporta anche la trasparenza delle immagini restituite nelle richieste, permettendo così la sovrapposizione visiva e la combinazione di più layer forniti da server geospaziali. Infine, se abilitato, consente agli utenti di ottenere informazioni specifiche su determinate caratteristiche presenti sulla mappa, come punti di interesse o dati geografici particolari.

Formato della richiesta

Lo standard specifica diversi tipi di richieste che possono essere effettuate ai server WMS:

- *GetCapabilities*: questa richiesta restituisce un documento XML che rappresenta le informazioni sul servizio WMS stesso, come il formato delle immagini delle mappe supportate, la compatibilità della versione del WMS e i layer disponibili, inclusi i dettagli come l'estensione geografica dei layer (bounding box), i sistemi di riferimento delle coordinate e l'opacità del layer.

- *GetMap*: utilizzata per ottenere un'immagine di mappa dal servizio. I parametri inclusi nella richiesta comprendono l'estensione geografica della mappa, il sistema di coordinate, lo stile di rendering e il formato dell'immagine desiderato.

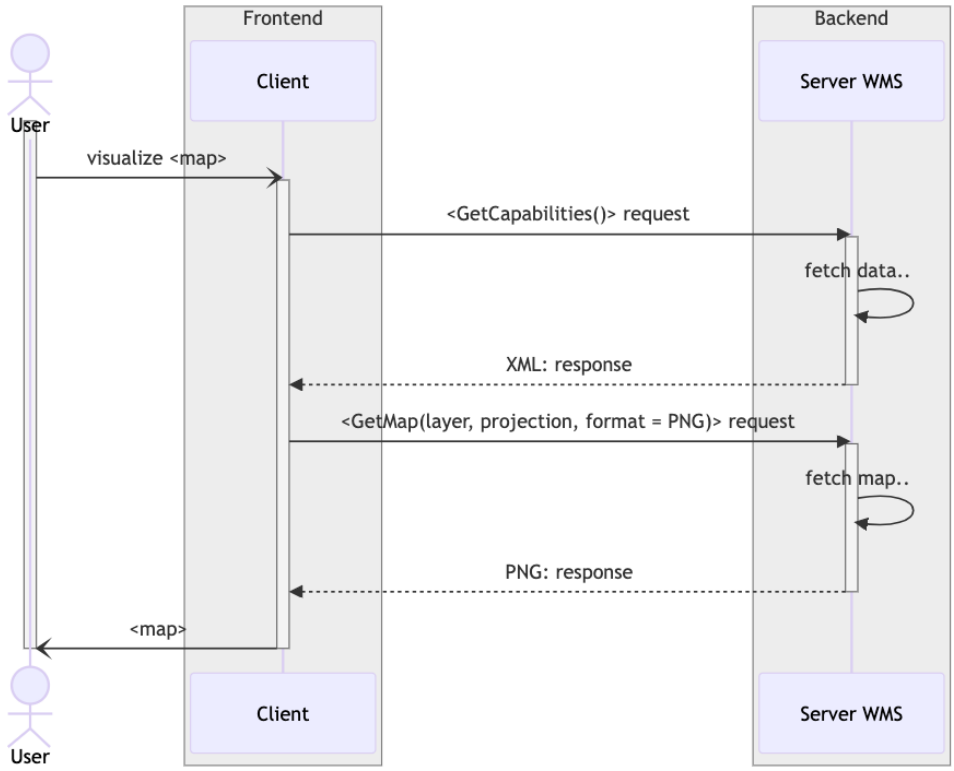

Figura 2.1: Diagramma di Sequenza del protocollo WMS

La risposta alla richiesta è un'immagine rasterizzata della mappa desiderata, pronta per essere visualizzata su un browser web o in un'applicazione client.

I tipi di richiesta che i fornitori di WMS possono opzionalmente supportare includono:

- *GetFeatureInfo*: Consente di ottenere informazioni dettagliate su specifiche caratteristiche presenti sulla mappa. Restituisce informazioni associate a quella posizione, come punti di interesse o dati geografici particolari.

- *DescribeLayer*: Restituisce i tipi di feature disponibili per i layer specificati, che possono essere ulteriormente descritti utilizzando richieste Web Feature Service (WFS). Questa richiesta è utile per comprendere meglio la struttura e le caratteristiche dei dati geospaziali presenti nel servizio WMS.

- *GetLegendGraphic*: Utilizzata per ottenere un'immagine della legenda della mappa, questa richiesta fornisce una guida visiva agli elementi presenti sulla mappa, come i simboli e le etichette dei layer.

Qui sotto è riportato uno *snippet* che contiene alcuni esempi possibili di richieste WMS, eseguite utilizzando il comando CURL per simulare il comportamento di una richiesta HTTP.

```
# Get Capabilities of the WMS service to retrieve metadata about available layers, formats, and operations.
curl -o GetCapabilitiesResponse.xml "http://example.com/wms?service=WMS&request=GetCapabilities"

# Get a map image of a specific region defined by the bounding box and other parameters.
curl -o GetMapResponse.png "http://example.com/wms?service=WMS&request=GetMap&layers=my_layer&width=800&height=600&bbox=10,20,30,40&format=image/png"

# Get Feature Info for a specific layer to obtain information about features at a particular location.
curl -o GetFeatureInfoResponse.xml "http://example.com/wms?service=WMS&request=GetFeatureInfo&layers=my_layer&query_layers=my_layer&info_format=text/xml&width=800&height=600&bbox=10,20,30,40&format=image/png&x=400&y=300"

# Describe the layer to retrieve detailed information about its properties and style.
curl -o DescribeLayerResponse.xml "http://example.com/wms?service=WMS&request=DescribeLayer&layers=my_layer&SLD_VERSION=1.1.0"

# Get the legend graphic for the layer to visualize the symbols used in the map legend.
curl -o GetLegendGraphicResponse.png "http://example.com/wms?service=WMS&request=GetLegendGraphic&layer=my_layer"
```

Qui di seguito invece viene mostrato al lettore un esempio di richiesta funzionante, fatta durante il percorso di tirocinio, ai server WMS del Geoportale Nazionale, gestito dal Ministero dell'Ambiente e della Sicurezza Energetica. Ciò serve per ottenere dati geospaziali relativi alla mappa che illustra l'area allagabile conforme al Piano di Gestione del Rischio di Alluvioni (PGRA) del 2021.

```
# Request to obtain the capabilities of the WMS service for the flood-prone area
curl -o GetCapabilitiesResponse.xml "http://wms.pcn.minambiente.it/ogc?map=/ms_ogc/WMS_v1.3/Vettoriali/Alluvioni_Estensione.map&service=wms&request=getCapabilities"
```

Come si può vedere dal messaggio di risposta in formato XML inviato dai loro Server, Il servizio offre funzionalità standard come la richiesta di mappe (GetMap) e informazioni (GetFeatureInfo), oltre a supportare la descrizione dei layer (DescribeLayer) e la visualizzazione della legenda (GetLegendGraphic).
Per ulteriori informazioni dettagliate sullo standard WMS, è possibile fare riferimento alla documentazione ufficiale disponibile sul sito web di Open Geospatial

Consortium [12]

2.1.2 Protocollo WMTS

A differenza del WMS, il quale invia un'unica immagine per ogni mappa ai client, il WMTS la "segmenta" in piccole porzioni rettangolari, denominate *tiles*.
Le tiles vengono prima pre-renderizzate e poi memorizzate sul server. Successivamente, sotto richiesta dei client, vengono recuperate e restituite come messaggio di risposta.
Quando un utente desidera visualizzare una mappa (nel nostro caso in un'applicazione web), il client invia tante richieste HTTP al server WMTS, una per ogni tile, specificando la porzione di mappa che vuole visualizzare, la proiezione e il formato dell'immagine desiderato. Il server, di conseguenza, risponde inviando le tiles corrispondenti alle richieste. Il client, appena ricevute le tiles, le combina insieme per formare la porzione di immagine effettiva di mappa richiesta.
Una delle principali caratteristiche del WMTS è che l'utilizzo delle tiles riduce la quantità di elaborazione richiesta dal server per mostrare correttamente la mappa. Il server WMTS deve infatti fornire solo le tiles necessarie per l'area richiesta, anziché generare e inviare un'unica immagine completa. Così facendo, si riduce sia il carico di rete, sia la quantità di elaborazione di dati che il server deve eseguire, rendendolo più efficiente rispetto al server WMS.
Inoltre, le tiles aprono le porte ad una politica di caching: trattandosi di piccole porzioni di immagini, queste possono essere facilmente memorizzate nella cache. Sul lato back-end, infatti, il servizio WMTS può cachare le tiles più utilizzate e restituirle più rapidamente al client. Lato front-end, invece, il browser è in grado di intercettare le risposte inviate dal server e di memorizzare nella sua cache interna l'immagine della tile. In questo modo, quando il client richiede la stessa tile, quest'ultima viene visualizzata senza dover contattare nuovamente il server. Questo è possibile poiché le tiles sono di piccole dimensioni, e quindi il peso effettivo della richiesta è sufficiente da poter memorizzare la risposta nella cache del browser. Questo consente di ridurre il carico di richieste sul server e di migliorare la velocità di caricamento delle mappe sui client.

Formato della richiesta

- *GetCapabilities*: Restituisce un documento XML che descrive il *Manifest* del servizio WMTS, inclusi i layers disponibili, i formati supportati, le coordinate di riferimento e altro ancora.

- *GetTile*: Restituisce una Tile specifica in base alle coordinate geografiche e allo zoom specificato nella richiesta.

- *GetFeatureInfo*: Questa richiesta non è molto comune nei servizi WMTS, in quanto generalmente viene direttamente utilizzata la richiesta GetFeatureInfo fornita dal servizio WMS. Se abilitata, questa richiesta restituisce informazioni

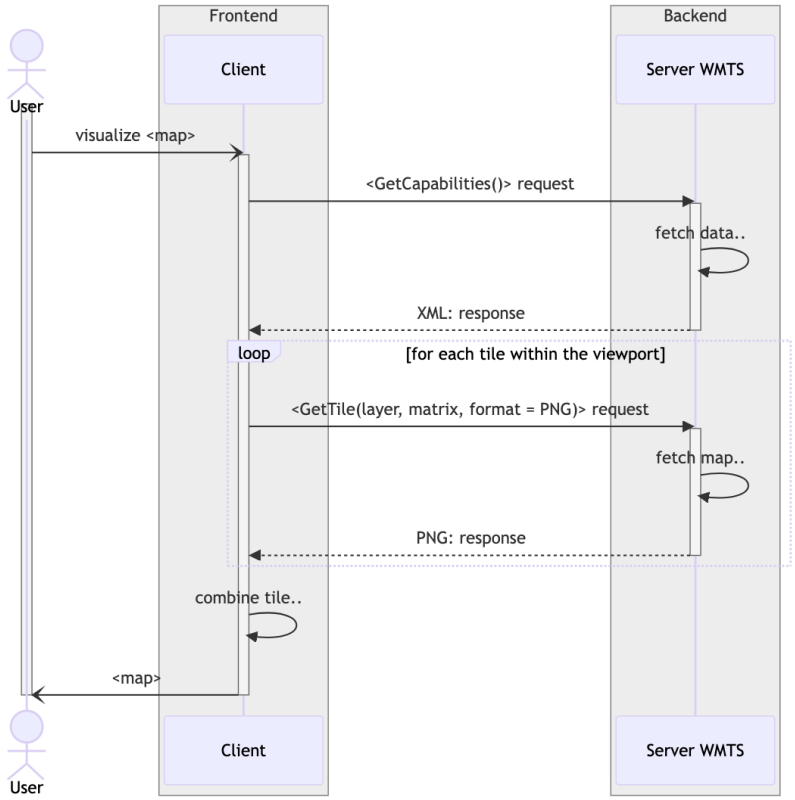

Figura 2.2: Diagramma di Sequenza del protocollo WMTS

aggiuntive associate a un tile, come attributi dei dati geospaziali presenti in quel punto della mappa.

Qui di seguito è riportato uno *snippet* contenente alcuni esempi possibili di richieste WMTS, eseguite utilizzando il comando CURL per simulare il comportamento di una richiesta HTTP.

```
# Get Capabilities of the WMTS service to retrieve metadata about available layers, tile matrix sets, and operations.
curl -o GetCapabilitiesResponse.xml "http://example.com/wmts/service?service=WMTS&request=GetCapabilities"

# Get a specific map tile for a given layer, tile matrix set, and tile coordinates.
curl -o GetTileResponse.png "http://example.com/wmts/service?service=WMTS&request=GetTile&layer=my_layer&tileMatrixSet=my_tile_matrix_set&tileMatrix=1&tileRow=0&tileCol=0&format=image/png"

# Get Feature Info for a specific layer to obtain information about features at a particular location.
curl -o FeatureInfoResponse.xml "http://example.com/wmts/service?SERVICE=WMTS&request=GetFeatureInfo&layer=mylayer&
```

```
tileMatrixSet=mytilematrixset&tileMatrix=1&tileRow=0&
tileCol=0&format=image/png&infoFormat=text/xml&I=50&J=50"
```

Per ulteriori informazioni dettagliate sullo standard WMTS, è possibile fare riferimento alla documentazione ufficiale disponibile sul sito web di Open Geospatial Consortium [12]

2.1.3 Protocollo WFS

Il Web Feature Service (WFS) è un servizio web utilizzato per accedere e manipolare dati geospaziali attraverso internet. La differenza principale tra un servizio WMS e WFS risiede nel tipo di dati che vengono trasmessi e nel modo in cui vengono rappresentati dal client.
Come già spiegato, il servizio WMS si occupa principalmente di offrire una rappresentazione visiva delle informazioni geografiche, distribuendo le mappe principalmente sotto forma di immagini PNG o JPG. Quest'ultime, essendo immagini rasterizzate, sono già pronte per essere visualizzate e non richiedono ulteriori elaborazioni: il client dovrà soltanto rappresentarle nella corretta proiezione.
Un WFS trasmette dati geospaziali grezzi, come numeri, coordinate e altre proprietà, che devono essere interpretati e visualizzati dal client. Ciò significa che sarà compito del client elaborare questi dati e rappresentarli graficamente sulla mappa, disegnando ad esempio punti, linee o poligoni secondo le specifiche della visualizzazione desiderata. Tali dati, che prendono il nome di "Feature", sono dati geospaziali vettoriali che vanno a rappresentare oggetti geografici reali come per esempio strade, fiumi o frane. Ogni feature possiede al proprio interno un insieme di attributi necessari, che ne vanno a definire il tipo di geometria e la posizione sulla mappa e inoltre possiede ulteriori proprietà facoltative che vanno a dare più informazioni sulla feature stessa, come per esempio nome, altezza, larghezza, regione, etc..
Per comprendere meglio la differenza, qui di seguito è riportato un esempio di dato geospazile, in formato GeoJSON che è stato utilizzato durante il tirocinio.

```
1  {
2    "type": "FeatureCollection",
3    "features": [
4      {
5        "type": "Feature",
6        "id": "frane_piff_opendata.fid--4f65597d_18e8ca7ffb022",
7        "geometry": {
8          "type": "Point",
9          "coordinates": [
10           426062.9254,
11           4940128.705
12         ]
13       },
14       "geometry_name": "geom",
```

```
15        "properties": {
16          "id_frana": "0042527700",
17          "tipo_movimento": 11,
18          "nome_tipo": "Aree con frane superficiali diffuse",
19          "regione": 1,
20          "nome_reg": "Piemonte",
21          "provincia": 4,
22          "nome_prov": "Cuneo",
23          "comune": 4196,
24          "nome_com": "Rodello",
25          "autorita_distretto": 2,
26          "nome_distr": "Po"
27        },
28        "bbox": [
29          426062.9254,
30          4940128.705,
31          426062.9254,
32          4940128.705
33        ]
34      }
35    ]
36  }
```

Similmente al WMS, che attraverso la richiesta GetMap può richiede una mappa specifica al server, il servizio WFS offre la richiesta GetFeature per richiedere una feature precisa che il server WFS provvederà a fornirgli.
I client possono anche eseguire operazioni di analisi spaziale, come il calcolo dell'area di un poligono o la ricerca delle feature più vicine a una determinata posizione. Ad esempio, è possibile interrogare il servizio per ottenere informazioni su tutti gli edifici situati in una determinata area, oppure trovare tutti i fiumi di una certa lunghezza. Queste informazioni vengono trasmesse utilizzando il formato GML (Geography Markup Language), ovvero un formato basato su XML che consente di descrivere in modo dettagliato la geometria e le proprietà di ciascuna feature. Ciò non toglie che i servizi WFS possano supportare anche altri tipi di formati, come il GeoJSON. In questo caso sarà dovere del server, definire all'interno delle informazioni presenti nella richiesta di GetCapabilities, tutti i formati geospaziali supportati. A quel punto il client, potrà richiedere il tipo di formato desiderato specificandolo nei queryParams (per esempio *outputFormat=application/json*).

Formato della richiesta

Le richieste che è possibile fare a un servizio WFS (Web Feature Service) sono le seguenti:

- *GetCapabilities*: Come per i WMS, la richiesta GetCapabilities restituisce un documento XML che fornisce informazioni dettagliate sul servizio WFS stesso. Queste informazioni includono i tipi di feature disponibili, i formati di trasmis-

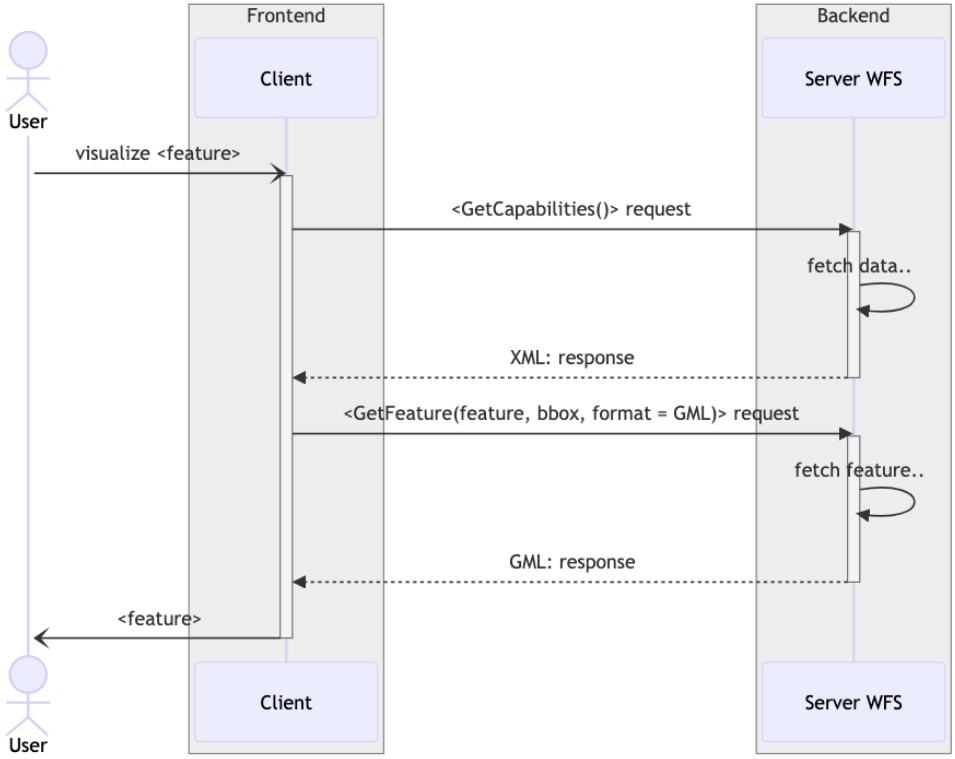

Figura 2.3: Diagramma di Sequenza del protocollo WFS

sione dei dati supportati (GML, GeoJSON, ecc..), i tipi di richieste supportate, le proiezioni geografiche disponibili e altre capacità del servizio.

- *GetFeature*: è la richiesta principale nel protocollo WFS in quanto consente agli utenti di recuperare la feature interessata. I parametri inclusi nella richiesta comprendono il nome della feature, il tipo di formato desiderato, la sua proiezione, ecc..

- *DescribeFeatureType*: è usata per richiede informazioni su un singolo tipo di feature prima di chiedere effettivamente la feature stessa. In particolare, l'operazione richiede un elenco di feature e attributi per il tipo di feature dato, oppure un elenco dei tipi di feature disponibili.

I tipi di richiesta che i servizi WFS possono opzionalmente supportare sono:

- *Transaction*: consente agli utenti di modificare i dati geografici nel servizio WFS, attraverso le operazioni come inserimento, modifica o eliminazione di una feature. Ciò è utile per gli utenti che hanno bisogno di mantenere aggiornati i dati geografici all'interno del servizio, consentendo loro di interagire direttamente e apportare modifiche in tempo reale.

- *GetPropertyValue*: questa richiesta consente agli utenti di ottenere i valori degli attributi di una o più feature, senza recuperarne tutti i dettagli geometrici associati.
- *LockFeature*: consente agli utenti di acquisire una lock di accesso in mutua esclusione alla feature, impedendo ad altri utenti di modificarla contemporaneamente.
- *GetFeatureWithLock*: questa richiesta combina le funzionalità di GetFeature e LockFeature. In questo modo è possibile acquisire una lock di accesso in mutua esclusione alla feature desiderate prima di richiedere la feature stessa.

Qui di seguito è riportato uno *snippet* contenente alcuni esempi possibili di richieste WFS, eseguite utilizzando il comando CURL per simulare il comportamento di una richiesta HTTP.

```
# Get Capabilities: Retrieve metadata about available layers, formats, and operations.
curl -o GetCapabilities.xml "http://example.com/wfs?service=WFS&request=GetCapabilities"

# GetFeature: Retrieve geographic features based on specific criteria.
curl -o GetFeature.xml "http://example.com/wfs?service=WFS&request=GetFeature&typeName=my_feature_type&bbox=10,20,30,40&outputFormat=GML"

# DescribeFeatureType: Get a detailed description of the structure of the available vector data.
curl -o DescribeFeatureType.xml "http://example.com/wfs?service=WFS&request=DescribeFeatureType&typeName=my_feature_type"

# Transaction: Make changes on geographic data in the WFS service.
curl -X POST -H "Content-Type: application/xml" --data-binary "@TransactionRequest.xml" "http://example.com/wfs?service=WFS&request=Transaction"

# GetPropertyValue: Get the attribute values of one or more features without retrieving the entire geometry
curl -o GetPropertyValue.xml "http://example.com/wfs?service=WFS&request=GetPropertyValue&typeName=my_feature_type&propertyName=my_property"

# LockFeature: Acquire exclusive feature access lock for update operations.
curl -X POST -H "Content-Type: application/xml" --data-binary "@LockFeatureRequest.xml" "http://example.com/wfs?service=WFS&request=LockFeature"
```

```
# GetFeatureWithLock: Get features with an exclusive access lock in a single operation
curl -o GetFeatureWithLock.xml "http://example.com/wfs?service=WFS&request=GetFeatureWithLock&typeName=my_feature_type&bbox=10,20,30,40&outputFormat=GML"
```

Ecco un altro esempio di richiesta funzionante, effettuata sempre durante il percorso di tirocinio, ai server WFS del Geoportale Nazionale. Come si può notare, questa richiesta è simile a quella effettuata precedentemente per il servizio WMS, ma questa volta viene rivolta al servizio WFS. Entrambi i servizi offrono informazioni sulla stessa mappa, ma con approcci diversi: mentre il WMS fornisce immagini della mappa, il WFS trasmette le feature geospaziali che compongono la mappa.

```
# Request to obtain the capabilities of the WFS service for the flood-prone area
curl -o GetCapabilitiesResponse.xml "http://wms.pcn.minambiente.it/ogc?map=/ms_ogc/wfs/Alluvioni_Estensione.map&service=wfs&request=getCapabilities"
```

Anche in questo caso, come si può vedere dal loro messaggio di risposta in formato XML inviato dai loro Server, il servizio offre funzionalità standard come la richiesta di feature (GetFeature) e la descrizione dei tipi di feature (DescribeFeatureType), oltre a supportare la modifica dei dati geografici tramite transazioni (Transaction). Per ulteriori informazioni dettagliate sullo standard WFS, è possibile fare riferimento alla documentazione ufficiale disponibile sul sito web di Open Geospatial Consortium [9]

2.1.4 Protocollo WCS

Infine, il Web Coverage Service (WCS) è uno standard creato per agevolare l'accesso e l'interrogazione di dati raster, comunemente denominati "coperture" o "coverage". A differenza dei servizi convenzionali che forniscono solo immagini pre-renderizzate (come nel caso dei WMS), il WCS attraverso le coverage permette agli utenti di ottenere porzioni di immagini, divise in segmenti. Questi segmenti possono essere considerati come blocchi di dati geospaziali che rappresentano parti di una mappa o di un'immagine geografica più ampia. Le coverage semplificano quindi l'accesso ai dati, consentendo agli utenti di richiedere solo le porzioni di interesse senza dover scaricare l'intera immagine.
Inoltre, i servizi WCS possono anche comprendere informazioni aggiuntive come metadati e altre informazioni contestuali.

2.2 Tiled Map Service (TMS)

Il protocollo Tiled Map Service (TMS), è un altro standard utilizzato per servire mappe geospaziali. A differenza di quelli sopra citati, questa specifica non è definita dall'Open Geospatial Consortium, ma è stata introdotta dall'Open Source Geospatial Foundation (OSGeo). Per questo motivo, nonostante entrambi i protocolli si occupano di fornire dati di mappe, presentano alcune differenze.
Questo standard può essere considerato come una semplificazione del servizio WMTS, in quanto continua a servire mappe geospaziali sotto forma di tiles, ma offre contemporaneamente meno funzionalità. Infatti questo standard non richiede che il client debba prima ottenere informazioni sul servizio attraverso una richiesta simile a *GetCapabilities*. Il client, quindi, può dedurre le informazioni sul servizio TMS dalla struttura dell'URL e può accedere direttamente alle tiles della mappa tramite percorso URL, senza dover chiedere alcuna informazione. Possiamo dunque affermare che la struttura degli URL di un servizio TMS segue una convenzione simile a quella delle cartelle: ogni livello di zoom della mappa e ogni tile è rappresentata da un URL univoco e questi possono essere organizzati in modo gerarchico, nello stesso modo delle cartelle in un File System.
Questo comporta da un lato un protocollo più semplice, ma dall'altro meno funzionalità: per esempio, il servizio non può fornirti informazioni su tutti i tipi di richieste a cui può essere interrogato. Nel caso di un servizio WMS, infatti, il client attraverso la GetCapabilities può capire o meno se il server supporta ulteriori richieste come GetFeatureInfo o GetLegendGraphic. Inoltre, il protocollo TMS prevede solo la trasmissioni di immagini tile, non restituisce altri tipi di informazioni (come nel caso della GetFeatureInfo che restituisce informazioni aggiuntive insieme all'immagine di mappa).
Ciò può sembrare una cosa negativa, ma è importante tenere conto anche del peso delle richieste. Una richiesta di getCapabilities ad un servizio di mappe può restituire un messaggio di risposta molto grande e pesante, sopratutto se contiene molti layer (mappe) al suo interno. Durante il tirocinio è capitato per esempio di ricevere risposte di un peso oltre i 30MB. Anche se può sembrare insignificante, gestire un peso simile all'interno di servizi web risulta molto tedioso, specialmente dal punto di vista della memorizzazione nella cache del browser.
Inoltre, avere una struttura gerarchica simile a quella di un File System, può portare a tanti benefici. Uno di essi, ad esempio, è la facilità nel distribuire tutte le tiles all'interno di una CDN (Content Delivery Network), La stessa cosa può essere fatta anche nel caso dei servizi WMTS tramite query params, ma risulta molto più complesso e difficile da gestire. L'utilizzo di una CDN, come per esempio i bucket S3 di Amazon, può migliorare significativamente le prestazioni, l'affidabilità e la scalabilità del servizio. In questo modo si ha un miglior bilanciamento del carico sul server, si può offrire la stessa mappa su più istanze contemporaneamente e viene garantita la continuità del servizio.
Infine, le richieste di un servizio TMS sono più facili da memorizzare nella cache del

browser rispetto al WMTS perché i suoi URL sono semplici e diretti. Al contrario, il WMTS spesso utilizza parametri più complessi nelle sue richieste, attraverso l'utilizzo dei query parameters, rendendo la memorizzazione nella cache del browser meno diretta e più soggetta a variazioni.
La struttura di una richiesta TMS per accedere ad una specifica tile è la seguente

```
curl -o tile.png "https://example.com/tms/1.0.0/my_map@EPSG
    :4326@png/{z}/{x}/{y}.png"
```

Questa richiesta restituisce direttamente una tile della mappa, generalmente grande 256x256, nel formato e proiezione richiesti. Le coordinate x e y rappresentano la posizione della tile all'interno della griglia, mentre la z il suo livello di zoom. Per ogni livello di zoom sono presenti dei confini, chiamati "bounding box" o "extent" che indicano l'estensione geografica delle tiles disponibili per quel livello di zoom. Se vengono richieste delle tile al di fuori di questi confini, il server TMS restituisce ovviamente un messaggio di errore.

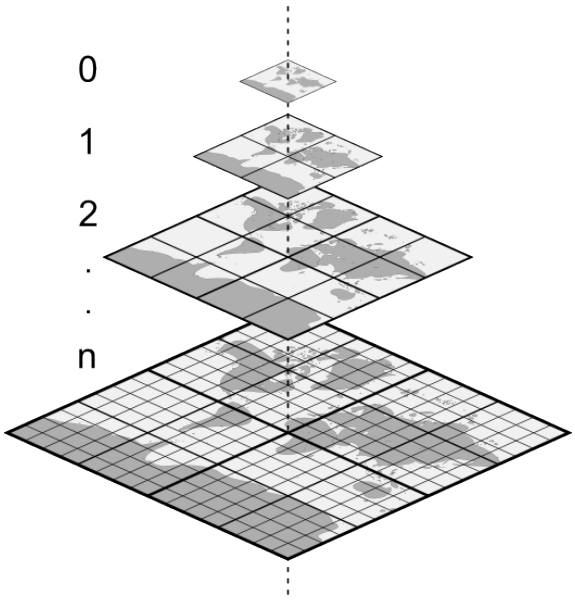

Figura 2.4: Tiles con livello di zoom

Formato della richiesta

Come indicato nella specificata del servizio TMS [7], esistono anche altre richieste che un client può effettuare per ottenere ulteriori informazioni oltre a quella relativa alle tiles. Le principali sono:

- *TileMapServiceResource*: questa richiesta è quella più simile a getCapabilities, permette di vedere tutte le mappe disponibili che il servizio TMS ha a disposizione, specificando il nome della mappa, il formato dell'immagine e la sua

proiezione. Può anche essere fornita più volte la stessa mappa, con formati e proiezioni differenti. Un esempio di risposta a tale richiesta è la seguente:

```
<TileMapService version="1.0.0" services="https://example.com">
<Title>Tile Map Service</Title>
<Abstract>A Tile Map Service Abstract</Abstract>
    <TileMaps>
    <TileMap title="my_map" srs="EPSG:4326" profile="local" href="https://example.com/tms/1.0.0/my_map@EPSG:4326@png"/>
    <TileMap title="my_map" srs="EPSG:4326" profile="local" href="https://example.com/tms/1.0.0/my_map@EPSG:4326@jpeg"/>
    <TileMap title="my_map" srs="EPSG:900913" profile="local" href="https://example.com/tms/1.0.0/my_map@EPSG:900913@png"/>
    <TileMap title="my_map" srs="EPSG:900913" profile="local" href="https://example.com/tms/1.0.0/my_map@EPSG:900913@jpeg"/>
    </TileMaps>
</TileMapService>
```

- *TileMapResource*: questa richiesta fornisce informazioni più dettagliate su una mappa, specificandone i confini, la dimensione delle tile, la proiezione e tutti i livelli di zoom disponibili che il client può richiedere (coordinata z) con i suoi rispettivi confini (extent). Un esempio di risposta a tale richiesta è la seguente:

```
<TileMap version="1.0.0" tilemapservice="https://example.com/tms/1.0.0">
<Title>my_map</Title>
<Abstract>A My Map Abstract</Abstract>
<SRS>EPSG:4326</SRS>
<BoundingBox minx="0.0" miny="0.0" maxx="90.0" maxy="90.0"/>
<Origin x="0.0" y="0.0"/>
<TileFormat width="256" height="256" mime-type="image/png" extension="png"/>
<TileSets profile="local">
    <TileSet href="https://example.com/tms/1.0.0/my_map@EPSG:4326@png/0" units-per-pixel="0.703125" order="0"/>
    <TileSet href="https://example.com/tms/1.0.0/my_map@EPSG:4326@png/1" units-per-pixel="0.3515625" order="1"/>
    <TileSet href="https://example.com/tms/1.0.0/my_map@EPSG:4326@png/2" units-per-pixel="0.17578125" order="2"/>
```

```
12     <TileSet href="https://example.com/tms/1.0.0/
     my_map@EPSG:4326@png/3" units-per-pixel="0.087890625"
     order="3"/>
13     <TileSet href="https://example.com/tms/1.0.0/
     my_map@EPSG:4326@png/4" units-per-pixel="0.0439453125"
     order="4"/>
14 </TileSets>
15 </TileMap>
```

- *TileResource*: la richiesta effettiva per richiedere la tile che corrispondete esattamente a quella spiegata in precedenza.

Il ciclo di vita di un client che richiede una mappa da un server TMS può essere descritto nel seguente modo: Inizialmente, il client contatta tramite richiesta TileMapServiceResource un server TMS per ottenere le informazioni di tutte le mappe disponibili. Dopo aver selezionato la mappa desiderata, il client utilizza la richiesta TileMapResource per ottenere tutti i possibili livelli di zoom, con i relativi confini. A questo punto il client richiede direttamente le tiles della mappa desiderata attraverso le coordinate x, y e z, senza sforare dai confini. Il server risponde risponde con l'immagine della tile.

2.3 Formati di file GeoSpaziali

I formati di file geospaziali sono tipi di file utilizzati per memorizzare e scambiare dati geografici e spaziali. Questi dati possono includere informazioni come mappe, immagini satellitari, dati di altitudine, di utilizzo del suolo e molti altri. Alcuni dei formati di file geospaziali più comuni includono:

- *Shapefile* (.shp): è uno dei formati più diffusi per memorizzare dati geografici vettoriali, come punti, linee e poligoni. È stato sviluppato da ESRI e può essere utilizzato con una varietà di software GIS.

- *GeoTIFF* (.tif): è una versione del formato di file TIFF (Tagged Image File Format) che include dati georeferenziati. È comunemente utilizzato per memorizzare immagini raster geograficamente riferite.

- *GeoJSON* (.json): è un formato di file basato su JSON (JavaScript Object Notation) utilizzato per memorizzare dati geospaziali vettoriali. È spesso utilizzato per lo scambio di dati geografici su web e tra diversi software GIS.

- *KML/KMZ* (.kml, .kmz): KML (Keyhole Markup Language) è un formato di file XML utilizzato per memorizzare dati geografici, come punti di interesse e percorsi. KMZ è una versione compressa di KML che include anche le risorse associate, come immagini e modelli 3D.

- *GeoPackage* (.gpkg): è un formato di file standard OGC (Open Geospatial Consortium) per memorizzare dati geospaziali in un singolo file SQLite. Supporta dati vettoriali e raster e può includere anche metadati e stili. Rappresentano dei database geografici.

La diversità di formati esistenti non costituisce un problema per i client che necessitano di utilizzare dati geospaziali. Normalmente, si tende a conservare dati sui server nei formati di cui sopra (principalmente, nel tirocinio, sono stati utilizzati Shapefile e GeoJson). Un server può supportare contemporaneamente più servizi (WMS, WMTS, WFS, TMS, etc..). Questo perché ciascuno di essi è in grado di convertire gli stessi dati di mappa iniziali in formati diversi, solitamente quelli richiesti dai client. Ad esempio, WMS permette di richiedere una mappa in formato PNG, JPG e GIF. Quando viene effettuata una richiesta tramite questo servizio, questi accede ai dati conservati, ad esempio, in Shapefile, e li converte nel formato specificato. è da notare come non ci si debba necessariamente attenere ad uno dei servizi standard esistenti, ma si possa anche specificarne uno custom che permetta di restituire i dati iniziali in un formato scelto a proprio piacimento. Il caso più comune è che si vadano restituire i dati nel loro formato originario.

2.4 OpenLayers

OpenLayers è una libreria open source JavaScript che fornisce strumenti per la visualizzazione di mappe interattive su pagine web. Fornisce grande capacità di astrazione per gli sviluppatori, andando ad incapsulare tutte le funzionalità dei servizi menzionati fino ad ora.

Caratteristiche Principali

Una delle caratteristiche principali di OpenLayers è la sua capacità di offrire un interfaccia grafica di mappe omogenea su diversi tipi di dispositivi e browser. Essa permette, in modo intuitivo e coinciso, di accedere a una serie di funzionalità basilari, quali zoom, capacità di trascinare la mappa e interazione con elementi a schermo (come marker o punti).
Offre inoltre API ad alto livello che consentono di integrare facilmente diversi tipi di servizi geospaziali nei progetti, come WMS, WMTS, WFS, WCS, TMS etc.. Sono supportati anche formati strettamente geospaziali, tra cui GeoJSON, Shapefile, KML, GPX, etc.. L'accesso a questi dati è possibile sia contattando un server esterno, senza aver bisogno di utilizzare esplicitamente i servizi sopra citati, sia salvando i relativi file direttamente il locale per poi accedervi. Tutti i sistemi di coordinate geografiche e proiezioni cartografiche sono disponibili in questa libreria, evitando di doverli calcolare da zero.
Infine, OpenLayers supporta di default alcuni provider di mappe, tra cui: OpenStreetMap, Bing Maps, Google Maps, MapQuest etc.. Ciò non preclude la possibilità di utilizzare altri servizi di mappe esterni.

Architettura di OpenLayer

Essendo una libreria OpenSource, il codice di OpenLayers è interamente disponibile su Github [6].
Inoltre, come si può vedere dalla documentazione [5], l'architettura di OpenLayers è basata su un modello di programmazione modulare, in cui ogni modulo svolge precise funzionalità per la creazione e la gestione delle mappe. I moduli di OpenLayer principali sono:

- ol/Map: è il core della libreria. Esso rappresenta la mappa stessa e fornisce funzionalità di base come la gestione dei layer, degli eventi e delle view.
- ol/View: questo modulo si occupa di gestire la View (la vista) della mappa, controlla la proiezione e il livello di zoom della mappa. La vista determina cosa viene mostrato sulla mappa e in quale modo.
- ol/layer: si occupa della creazione del layer sulla mappa, inclusi layer vettoriali, raster e di tile.
- ol/source: fornisce classi per definire le fonti di dati per i layer, ad esempio fonti WMS, WFS, GeoJSON, ecc.
- ol/control: viene utilizzato per la creazione di controlli e widget per la mappa, come strumenti per lo zoom, la scala, la misurazione delle distanze, ecc.
- ol/interaction: si occupa dell interazioni utente sulla mappa, come la selezione di una feature, lo zoom effettuato attraverso il dopppio click o lo scroll del mouse e altre azioni simili.
- ol/proj: Questo modulo gestisce la gestione dei sistemi di coordinate e delle proiezioni cartografiche.

OpenLayers supporta anche l'aggiunta di plugin di terze parti per estendere le sue funzionalità. Una raccolta di plugin è fornita sul loro sito al seguente link.

2.5 GeoServer

GeoServer è un software OpenSource scritto in Java che permette di caricare e servire dati di mappe geospaziali. All'interno di GeoServer i dati possono essere caricati in vari formati (come Shapefile, GeoPackage, etc..) per poi essere esposti attraverso l'utilizzo di protocolli standard OGC (come WMS, WFS, etc..), oppure attraverso l'utilizzo di altri protocolli come TMS. Ad esempio, è possible caricare un file di mappa in formato Shapefile ed esporlo sul proprio server mediante un servizio WMS. In questo modo un client che supporta tale protocollo (per esempio OpenLayer), può visualizzare la mappa interessata e richiederla in un formato raster come PNG, JPG o simili.

Caratteristiche Principali

Una funzionalità del GeoServer è il "mirroring" di servizi OGC, ovvero consente di replicare e mantenere aggiornati i servizi di mappe forniti da server esterni, i quali espongono i dati geospaziali tramite protocolli WMS, WFS, etc..
Ciò è possibile in quanto il GeoServer è in grado di memorizzare al suo interno l'URL che rappresenta la richiesta di *GetCapabilities*. Come già spiegato, questa richiesta è essenziale per ottenere tutte le informazioni necessarie (layer di mappe, proiezioni, formati, etc..) al fine di richiedere le mappe fornite dal server esterno.
In pratica, ogni volta che un client richiede al GeoServer una mappa che non è presente localmente, il GeoServer inizia contattando il server esterno tramite una richiesta di GetCapabilities. Una volta ricevuta la risposta, il GeoServer esegue un'altra richiesta (ad esempio una GetMap nel caso dei servizi WMS) per ottenere la mappa specifica richiesta dal client. Una volta che il GeoServer riceve i dati della mappa dal server esterno, li invia al client come risposta alla sua richiesta originale.
Questo consente di memorizzare tutti i dati delle mappe, potenzialmente anche esposte mediante protocolli diversi, sotto un unico server. Tale pratica non solo semplifica la gestione e la manutenzione dei dati geospaziali, ma dal punto di vista del client, viene offerta l'illusione che tutte le mappe siano contenute direttamente su un unico server.
Un'altra importante funzionalità di GeoServer è la capacità di esporre dati geospaziali utilizzando protocolli diversi. Ad esempio, è possibile registrare un servizio esterno di mappe WMS nel modo spiegato precedentemente per poi fornirlo come servizio WMTS o TMS, aumentandone così le prestazioni. In automatico il GeoServer si occuperà di fornire i della mappa in tiles, nonostante il servizio esterno non supporti questo formato. Questa caratteristica è particolarmente vantaggiosa per il Fron-tend poiché elimina la necessità di supportare tutti i formati di dati.

Meccanismi di Caching

Avere tutte le mappe centralizzate sotto un unico server favorisce anche l'utilizzo di meccanismi di caching: a livello di WebServer, per esempio, tramite l'utilizzo di un Reverse Proxy, è possibile configurare politiche di caching per memorizzare temporaneamente le risposte alle richieste HTTP inviate dal GeoServer. Questo consente di ridurre il carico su di esso, in quanto alcune richieste possono essere soddisfatte direttamente dalla cache del WebServer senza doverle inoltrare nuovamente al GeoServer. Tale pratica risulterebbe più complessa da fare, se tutte le mappe fossero decentralizzate su servizi esterni, specialmente se di terze parti.
Inoltre, GeoServer stesso offre diversi livelli di caching, è possibile infatti configurarlo per memorizzare in una sua cache interna le risposte alle richieste di mappe.
Esiste anche la possibilità di memorizzare nella cache la risposta ottenuta dalla richiesta GetCapabilities inviata ai servizi OGC esterni, in modo da evitarne il recupero ogni volta. Questo risulta utile soprattutto quando si trattano mappe geospaziali che cambiano raramente nel tempo o che non cambiano affatto, anche se è possibile

comunque risolvere il problema in parte mettendo una cache di durata più breve. Questo consente a GeoServer di rispondere più rapidamente alle richieste di mappe, utilizzando i dati già memorizzati nella cache anziché elaborarli nuovamente, riducendo così il carico e migliorando i tempi di risposta.

Architettura di GeoServer

Geoserver supporta diversi tipi formati e servizi geospaziali, i più noti sono:

- Servizi di mappe: WMS, WMTS, WFS, WCS, TMS per i servizi di mappe;
- Dati raster: GeoPackage, ArcGrid, GeoTIFF, ImageMosaic, WorldImage;
- Dati vettoriali o a oggetti geospaziali: ShapeFiles, GeoPackage, PostGIS;

Questi ultimi possono essere caricati e gestiti tramite l'utilizzo di un'interfaccia grafica, realizzata sotto forma di un'applicazione web, oppure tramite l'utilizzo di chiamate ad una REST API.
Al fine di poter caricare correttamente una mappa all'interno di GeoServer, è necessario creare tre componenti basilari:

- Workspace: rappresenta precisamente l'"area di lavoro", cioè il luogo in cui tutte le mappe vengono effettivamente caricate, quest'ultime a sua volta vengono organizzate in all'interno di Storage. Una workspace può contenere uno o più Storage.
- Storage: rappresenta il tipo di servizio di mappa che si vuole caricare all'interno di GeoServer. Può essere un servizio WMS, WFS, WCS, o un tipo di dato geospaziale come Shapefile, GeoTIFF, etc.. Ovviamente in base al tipo di Storage che si va a creare, le impostazioni visualizzate sull'interfaccia grafica (o i query param da inviare secondo la rest API) saranno differenti. Per esempio, se si va a creare un servizio esterno WMS, sarà chiesto il link delle capabilities da contattare. Se si andrà a caricare uno Shapefile, sarà richiesto il percorso locale in cui il file si trova.
- Layer: rappresenta la mappa effettiva, quella che verrà poi fornita al Front-end. È ciò che restituisce lo storage una volta ricevuta la risposta alla getCapabilities, nel caso di un servizio OGC, o ciò che contiene uno Shapefile. Ovviamente più layer possono stare all'interno di uno storage.

Dal punto di vista di architetturale, come si può vedere su Github [4], GeoServer è stato sviluppato utilizzando Spring come Framework Java, e viene utilizzato per gestire tutte le funzionalità sopra citate. Anche la REST API è stata implementata sempre tramite Spring (Spring MVC Framework), mentre l'interfaccia grafica è stata realizzata invece tramite l'utilizzo di Wicket.

Tutta via, come viene riportato dagli sviluppatori di GeoServer nella loro documentazione [3], la REST API presenta alcune criticità importanti che possono influenzare il suo utilizzo e la sua affidabilità. Innanzitutto, la specifica della REST API è rimasta alla versione di Swagger 2.0 e non è aggiornata allo al nuovo standard di OpenAPI 3.0, ciò comporta ad una peggiore comprensione e utilizzo dell'API stessa. Ma sopratutto, viene fatto notare che i file di Swagger utilizzati per documentare l'API sono stati creati manualmente nel 2017 e non sono stati più aggiornati con l'evoluzione dell'infrastruttura del software.
L'utilizzo dell'API di GeoServer è stata una parte cruciale durante periodo di tirocinio, e questo problema ha rappresentato una sfida significativa, in quanto è stato necessario sistemare manualmente le richieste al fine di poterle utilizzare correttamente.

Capitolo 3

Introduzione al Tirocinio

3.0.1 Introduzione al Problema

Come già anticipato nell'introduzione, la parte delle mappe all'interno dell'applicazione era completamente assente, in quanto gli sviluppatori erano dedicati alla realizzazione di altre funzionalità, anch'esse prioritarie. Infatti, il candidato, insieme alla supervisione del tutore aziendale, si sono occupati di valutare e scegliere le migliori tecnologie da utilizzare per implementare tale aspetto.
Dal punto di vista del Back-end, l'applicazione non presentava ancora nessun meccanismo di gestione e fornitura dei dati geospaziali, mentre nel Front-end, lo sviluppo delle mappe era appena cominciato, con la realizzazione di un piccolo componente in Angular. Tale componente possedeva solamente l'integrazione agli elementi base di OpenLayers, un supporto alla sovrapposizione multipla di Layer differenti e un layer di default, che mostrava la mappa fornita da OpenStreetMap.

3.0.2 Cosa è stato Realizzato

Capitolo 4

Tecnologie Aziendali Utilizzate

All'inizio dell'esperienza di tirocinio, il tutor aziendale si è occupato di fornire una panoramica generale sulle tecnologie e sui framework utilizzati all'interno dell'azienda.

4.0.1 Microservizi e Docker

Dal punto di vista architetturale, è stata scelta un'architettura a microservizi anziché una monolitica. Essa prevede di suddividere l'applicazione in tanti servizi autonomi, ciascuno dei quali svolge una funzione specifica. Questo approccio è molto comune per le applicazioni di grandi dimensioni, poiché ogni microservizio può essere sviluppato e mantenuto in modo indipendente dagli altri, utilizzando, se necessario, anche linguaggi di programmazione differenti. Tale sistema permette agli sviluppatori di concentrarsi su ogni funzionalità singolarmente, rimanendo isolati da tutto il resto dell'applicazione. Inoltre, grazie alla possibilità di utilizzare linguaggi differenti, è possibile scegliere la soluzione più adatta in base alla funzionalità che deve essere implementata. Quest'architettura favorisce la scalabilità orizzontale, ovvero la possibilità di aggiungere o rimuovere istanze di singoli microservizi in base alle esigenze di carico, senza dover scalare l'intera applicazione. Dunque, rende più efficiente l'utilizzo delle risorse e consente di gestire picchi di traffico in modo più efficace.
Per gestire questo tipo di architettura, si è scelto di utilizzare Docker, ovvero una piattaforma che consente di creare e gestire container, cioè ambienti isolati che contengono tutte le risorse necessarie per eseguire un'applicazione. Ogni servizio viene dunque "containerizzato", ovvero viene isolato in un ambiente autonomo con tutte le risorse e le dipendenze necessarie per il suo funzionamento. La containerizzazione attraverso Docker rende notevolmente più semplice lo sviluppo dell'applicazione, in quanto gli sviluppatori, lavorando sullo stesso container, possiedono tutti lo stesso ambiente di sviluppo: la stessa versione delle dipendenze, librerie e del linguaggio di programmazione. Inoltre, una volta che l'applicazione è pronta per il deployment (cioè pronta per essere messa in produzione), viene eseguita su server in un ambiente ancora containerizzato. Così facendo, si ha la garanzia che l'applicazione continui

a funzionare come quando è stata sviluppata, e, se necessario, è possibile scalarla orizzontalmente attraverso l'uso di servizi Cloud come Amazon AWS.

4.0.2 WebServer e Reverse Proxy

È stato impiegato un WebServer che, tra le altre cose, ha svolto il ruolo di un Reverse Proxy, permettendo di semplificare la gestione del traffico HTTP diretto verso i container Docker. Un WebServer è un software che riceve tutte le richieste HTTP inviate da client (per esempio browser) e che alla fine fornirà loro delle risposte, solitamente sotto forma di dati o pagine web.
Comportandosi da reverse proxy, il WebServer reinstrada le richieste ricevute verso il microservizio appropriato, mascherando ai client la pluralità di servizi esistenti. Una volta ricevuta la risposta dal microservizio, la trasmette al client che ne aveva fatto richiesta. Nel dettaglio è stato utilizzato Caddy come WebServer, poiché offre delle semplici modalità di configurazione, tramite la stesura del file "Caddyfile", fornito dal software stesso.
Il suo uso principale è stato quello di permettere ai client di contattare i servizi containerizzati tramite un dominio, anziché un indirizzo ip con una porta associata: ad esempio, invece di contattare il container che ha al suo interno il servizio di rest API all'indirizzo *127.0.0.1:8080*, è possibile contattarlo tramite il dominio *bms.local.gkops.net/api/*. Grazie a questa modalità, è possibile simulare un ambiente di produzione, nel quale tutti i servizi vengono contattati su un dominio apposito tramite protocollo HTTPS. Quest'ultimo dettaglio è possibile grazie a Caddy che permette di installare certificati TLS sui domini da lui creati. È stato utilizzato Let's Encrypt come Certification Authority.
Inoltre, se necessario, tramite Caddy è possibile eseguire load balancing tra i microservizi, il quale consente di distribuire il carico tra più istanze del servizio per garantire una migliore scalabilità e affidabilità del sistema.
Infine, l'utilizzo di Caddy è essenziale per effettuare meccanismi di caching tra le richieste HTTP inviate dai client ai microservizi: nel caso in cui venga posta la medesima richiesta, il WebServer non richiederà nuovamente al il microservizio la risposta, bensì risponderà direttamente al client.

4.0.3 Front-end e Back-end

Per la parte di front-end, nel contesto di applicazioni Enterprise, come nel caso di BMS, è stato deciso di sviluppare l'interfaccia grafica utilizzando un framework JavaScript moderno, nello specifico Angular. Questi framework presentano un'architettura basata su componenti, cioè permettono di costruire l'interfaccia grafica attraverso l'uso di componenti che possono essere riutilizzati. Questo consente di avere una programmazione modulare, in cui ogni componente HTML viene visto come un modulo a se stante, che può essere importato e riutilizzato su pagine web differenti. Questo rende il codice più organizzato, facile da comprendere e da mantenere, specialmente all'aumentare delle dimensioni e della complessità dell'applica-

zione.
Lato back-end, invece, è stato adottato l'utilizzo di un framework ad alto livello come .NET, il quale offre un insieme di funzionalità e strumenti per lo sviluppo di applicazioni complesse e scalabili.

4.0.4 Continuous Integration e Delivery

- Github: per la manutenzione e revisione del codice;
- Teamcity per i test test cases, continuous integration e continuous delivery;

Capitolo 5

Svolgimento del Tirocinio

In questo capitolo verrà spiegato nel dettaglio lo svolgimento del tirocinio. L'obiettivo preposto è stato quello di realizzare un'applicazione funzionante per visualizzare e gestire mappe geospaziali. Lo sviluppo si è diviso tra front-end, per consentire la visualizzazione delle mappe, e back-end, per la loro gestione e fornirle al primo. Parte del codice si è inoltre focalizzata sulla comunicazione tra front-end e back-end.

5.1 Preparazione al Tirocinio

5.1.1 Studio di Reactive Programming e Angular

I primi giorni di tirocinio sono stati dedicati allo studio autonomo dei linguaggi di programmazione e delle librerie utilizzate all'interno del progetto. Particolare attenzione è stata dedicata inizialmente alla parte di sviluppo front-end, dedicando le prime settimane allo studio del framework JavaScript Angular, e al suo corrispettivo linguaggio di programmazione TypeScript. Lo studio di queste tecnologie è terminato con la realizzazione di un piccolo progetto in Angular 17, che è possibile replicare seguendo la sua documentazione [8]. Si è poi proseguito con lo studio della libreria RxJS, fondamentale per l'uso di Angular, in quanto è parte integrante dello stesso. Tale libreria viene utilizzata internamente da Angular per gestire numerose operazioni asincrone attraverso l'uso di Stream, adottando il paradigma di "Reactive Programming" invece che "Event Driven".
Nel paradigma a eventi, viene scritto del codice per gestire un evento quando questo si verifica. In genere si definisce una funzione, nota come callback, che viene registrata su un certo evento. Quando tale evento si verifica, come ad esempio un click del mouse su un pulsante, la funzione registrata viene invocata per gestire l'evento di click. Nell'approccio reattivo, invece, viene scritto del codice per gestire uno Stream di dati che rappresenta una sequenza continua di eventi. Ad esempio, invece di gestire il singolo click del mouse, viene creato uno Stream che rappresenta i click che avvengono del tempo. Tale Stream può essere manipolato, filtrato o trasformato utilizzando degli operatori.

Angular utilizza RxJS per adottare quest'ultimo tipo di paradigma, facendo uso di tre elementi interdipendenti: l'Observable, gli Operator e gli Observer.
L'Observable è colui che fornisce i dati sullo Stream ed è colui che è oggetto dell'osservazione. Durante il fornimento dei dati, che avviene in maniera continua nel tempo, gli Operator si occupano di manipolarli. Gli Operator sono delle operazioni che permettono la manipolazione dei dati sullo Stream in maniera asincrona. Le operazioni più note sono la mappatura, il filtraggio, la combinazione e la trasformazione. L'Observer si trova alla fine della catena e si occupa di utilizzare il risultato dei dati elaborati.
Angular utilizza internamente gli Observable a basso livello per gestire i suoi componenti, mentre ne espone un'interfaccia semplificata ad alto livello per l'utilizzo da parte degli sviluppatori. Ad esempio, il routing delle pagine web e il modulo del form di input fanno uso degli Observable per monitorare e rispondere agli eventi di input dell'utente.

5.1.2 Realizzazione di un prototipo con OpenLayers

Lo sviluppo delle mappe da parte del tirocinante è iniziato con lo studio e l'utilizzo della libreria OpenLayers. Al fine di capirne meglio il suo funzionamento, il candidato ha deciso di non iniziare direttamente con lo sviluppo del progetto di tirocinio, ma ha proseguito con l'implementazione di una piccola applicazione esterna, priva del framework Angular e librerie di terzi, così da semplificarne il suo utilizzo. Come riportato dalla documentazione [1], OpenLayers si basa su quattro componenti principali:

- Map: per la manutenzione e revisione del codice;
- View per i test test cases, continuous integration e
- Source
- Layers

Qui sotto viene riportato una parte del codice del prototipo, così da comprendere meglio il funzionamento della libreria.

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/x-icon" href="https://
    openlayers.org/favicon.ico" />
6     <meta name="viewport" content="width=device-width, initial
    -scale=1.0" />
7     <title>Testing OpenLayers</title>
8   </head>
```

```
9   <body>
10    <div id="map"></div>
11    <script type="module" src="./main.js"></script>
12  </body>
13 </html>
```

Listing 5.1: Codice HTML

```
1 import './style.css';
2 import { Map, View } from 'ol';
3 import TileLayer from 'ol/layer/Tile';
4 import OSM from 'ol/source/OSM';
5 import TileWMS from 'ol/source/TileWMS.js';
6
7 new Map({
8   target: 'map',
9   view: new View({
10    center: [0, 0],
11    zoom: 2
12  }),
13  layers: [
14    new TileLayer({
15      source: new OSM()
16    }),
17  ]
18 });
```

Listing 5.2: Codice JavaScript

5.1.3 Studio di OpenLayers in Angular

Il candidato, infine, ha confrontato il suo lavoro con quello già presente all'interno del progetto realizzato in Angular, così da comprendere al meglio le differenze fra le due implementazioni e capire quali funzionalità fossero già state realizzate.
La loro versione presentava solamente l'integrazione agli elementi base di OpenLayers, un supporto alla sovrapposizione multipla di layer differenti e un layer di default, che mostrava una mappa di base fornita da OpenStreetMap. La differenza principale, essendo stata realizzata in Angular, è che la loro implementazione era stata fatta tramite l'uso di componenti, in questo modo poteva il componente della mappa poteva essere facilmente importato più volte all'interno delle varie pagine web dell'applicazione.
Inizialmente, infatti, tale componente era situato all'interno della pagina web dell'UiKit, ma allo sviluppatore di importare più volte il componente della mappa nell'applicazione. ovvero una pagina contente, a scopo dimostrativo, tutti i componenti grafici che sarebbero stati poi utilizzati all'interno dell'applicazione. L'uso dell'UiKit è una pratica comune, specialmente nel contesto di applicazioni Enterprise, in

quanto permette agli sviluppatori.
Ovviamente l'uso dell'Ui-Kit è reso possibile grazie ai framework JavaScript come Angular, la cui architettura modulare rende possibile l'esportazione dei componenti creati in altre pagine web dell'applicazione. Ad esempio, quando uno sviluppatore crea un componente nell'Ui-Kit, può facilmente importarlo e utilizzarlo all'interno delle varie pagine del sito, senza doverlo ricreare da zero ogni volta. Senza Angular, questa operazione sarebbe molto più laboriosa, dovendo riscrivere più volte lo stesso codice e quindi non risulterebbe più conveniente.

5.1.4 Censimento delle Mappe

Il tirocinio è poi continuato con lo studio e alla comprensione dei protocolli OGC e dei concetti fondamentali relativi ai dati geospaziali. In particolare, l'attenzione è stata concentrata sui protocolli WMS (Web Map Service) e WFS (Web Feature Service), poiché sono tra i più utilizzati e facili da distinguere. Questo ha permesso di comprendere al meglio il loro funzionamento e le principali differenze tra essi. Per quanto concerne i dati geospaziali, invece, lo studio si è dedicato principalmente ai formati GeoJSON e Shapefile, in quanto sono anch'essi formati geospaziali comuni e ampiamente utilizzati.
Dopo questa fase di apprendimento iniziale, il candidato ha collaborato con l'azienda per identificare le fonti da cui raccogliere le mappe necessarie, censendole in un documento dettagliato. Questo processo ha permesso di avere una panoramica completa delle risorse geospaziali disponibili e di comprendere meglio come organizzarle e utilizzarle nel contesto del mio tirocinio. Le mappe utilizzate all'interno del progetto provengono principalmente da tre fonti:

- *Geoportale Nazionale*, un portale gestito dal Ministero dell'Ambiente e della Sicurezza Energetica che contiene dati geospaziali forniti tramite protocollo OGC, quali WMS, WFS e WCS. É possibile reperire le mappe utilizzate dal loro sito [10].

- *IdroGEO*, una piattaforma gestita ISPRA (Istituto Superiore per la Protezione de la Ricerca Ambientale), che permette agli utenti di accedere e di scaricare dati di mappe, in formato Shapefile o GeoJSON, relativi al rischio idrogeologico in Italia. Precisamente fornisce informazioni sull'Inventario dei Fenomeni Franosi in Italia (IFFI), e sulle mappe nazionali di pericolosità da frane e alluvioni e sugli indicatori di rischio associati.

- *GeoCart*, un'azienda italiana specializzata nella produzione e distribuzione di mappe e prodotti cartografici [11]. Anche in questo caso le mappe sono in formato Shapefile, ma a differenza di quelle precedenti, non sono di dominio pubblico e non sono accessibili in modo gratuito. Sono risorse sviluppate ai fini commerciali.

5.2 Sviluppo Front-end con OpenLayers

5.2.1 Perché OpenLayers

Tale libreria è stata scelta, rispetto alle altre concorrenti, in quanto offre internamente molte funzionalità utili per la rappresentazione di questo tipo di mappe.
Come già spiegato nel secondo capitolo, OpenLayers offre un supporto già integrato ai protocolli OGC, essenziali per la rappresentazione di questi tipi di dati geospaziali. La maggior parte delle sorgenti di mappe fornite, infatti, sono trasmesse mediante protocolli OGC quali WMS, WFS e WCS. Con l'utilizzo di questa libreria, la comunicazione ai servizi OGC esterni risulta essere più semplice, in quanto non è necessario eseguire manualmente tutte le richieste imposte dallo Standard, e, successivamente, eseguire il parsing sul contenuto delle risposte in formato XML.
OpenLayers fornisce, a più livelli, delle classi che si occupano di eseguire queste operazioni al posto nostro, e attraverso il principio di Information Hiding, espongono allo sviluppatore solo il minimo necessario per eseguire correttamente la richiesta.
Per esempio, nel caso dell'implementazione del protocollo WMS, non sarà quindi necessario eseguire una richiesta di GetCapabilities e successivamente GetMap, al fine di ricevere l'immagine della mappa. Se il nome del layer è noto, basterà istanziare la classe ImageWMS che passandogli i parametri corretti, eseguirà automaticamente le operazioni necessarie al fine di ricevere l'immagine della mappa corretta. Qui sotto viene riportato un esempio di implementazione della classe ImageWMS, il quale è possibile reperire anche dalla documentazione di OpenLayer[2]

```
1  import ImageWMS from 'ol/source/ImageWMS.js';
2  import Map from 'ol/Map.js';
3  import OSM from 'ol/source/OSM.js';
4  import View from 'ol/View.js';
5  import { Image as ImageLayer, Tile as TileLayer } from 'ol/
     layer.js';
6
7  const layers = [
8    new TileLayer({
9      source: new OSM(),
10   },
11   new ImageLayer({
12     extent: [-13884991, 2870341, -7455066, 6338219],
13     source: new ImageWMS({
14       url: 'https://ahocevar.com/geoserver/wms',
15       params: { 'LAYERS': 'topp:states' },
16       ratio: 1,
17       serverType: 'geoserver',
18     },
19   },
20 ];
21 const map = new Map({
22   layers: layers,
```

```
23   target: 'map',
24   view: new View({
25     center: [-10997148, 4569099],
26     zoom: 4,
27   }),
28 });
```

Listing 5.3: Esempio di ImageWMS

Come si può vedere dall'esempio, al fine di recuperare l'immagine della mappa fornita dal server WMS esterno, è necessario istanziare la classe ImageWMS passandogli come argomento soltanto l'URL e il layer che si vuole visualizzare. OpenLayer si occuperà di eseguire la richiesta di GetMap al posto nostro e di mostrare a schermo l'immagine della mappa.
Tale esempio richiede ovviamente che il nome del layer sia noto, nel caso in cui il client non conosca la lista di layer disponibili, sarà comunque necessario eseguire una richiesta di GetCapabilities.
Anche in questo caso, OpenLayer si occupa di eseguire la richiesta e di effettuare il parsing in automatico sul contenuto dell'XML, senza dover creare una struttura dati apposita per manipolare i dati ricevuti da essa.

```
1 import WMSCapabilities from 'ol/format/WMSCapabilities.js';
2
3 const parser = new WMSCapabilities();
4
5 fetch('data/ogcsample.xml')
6   .then(function (response) {
7     return response.text();
8   })
9   .then(function (text) {
10    const result = parser.read(text);
11    document.getElementById('log').innerText = JSON.stringify(
    result, null, 2);
12  });
```

Listing 5.4: Esempio di GetCapabilities per WMS

Inoltre la libreria ha già un supporto integrato per il formato GeoJSON, il che la rende ottimale al fine di testare le mappe fornite da IdroGEO, in quanto quest'ultime non vengono distribuite mediante un servizio OGC, ma vengono fornite come file locali in formato GeoJSON.
Infine, la libreria è interamente supportata da Angular e con un semplice comando di npm (mpm install ol), la si può installare all'interno del progetto, il che la rende ottimale per questo tipo di applicazione.

5.2.2 Implementazione nel Progetto

Il candidato, sotto supervisione del tutor aziendale, ha iniziato lo sviluppo del progetto BMS creando all'interno dell'applicazione una nuova pagina web, nella quale ha importato il componente della mappa già esistente nell'Ui-Kit, così da poter integrare il supporto ai protocolli OGC richiesti e iniziare a testare le mappe censite. Lo sviluppo è proseguito estendendo il componente Angular delle mappe, che fino a quel momento ha presentato solamente l'integrazione agli elementi base di OpenLayers e un supporto alla sovrapposizione multipla di layer differenti.
Come prima funzionalità si è implementato il supporto al formato GeoJSON, selezionando come prima mappa da testare quella delle frane in Toscana, fornita da IdroGEO, in quanto tutte quelle fornite da questo ente, possono essere scaricate localmente e caricate all'interno del progetto, senza dover contattare alcun servizio esterno. Ciò facilità il lavoro di programmazione, riducendo il rischio di errore dovuto a eventuali problemi di disservizio dei server di mappe esterni.
In questa fase, le parti di codice che hanno subito più modifiche sono le classi BuildLayer.ts e MapModel.ts, le quali si occupano di far istanziare l'oggetto della mappa con proprietà differenti, in base al tipo di formato geospaziale passato come argomento.
Il lavoro è continuato con l'implementazione della pagina web descritta precedentemente, andando a inserire al suo interno alcune mappe fittizie, tra cui una in formato GeoJSON, così da poterne verificare il funzionamento. Tali mappe, rappresentate come "layer", sono state tutte sovrapposte a quella di default, ovvero quella di OpenStreetMap. In questo modo si è compreso se i layer sono stati rappresentati correttamente, riuscendo così a orientarsi all'interno delle mappe stesse in termini geografici.

spiegare openstreet map voglio dire che ho aggiunto il layer di openstreet map voglio dire del problema di pro4j voglio dire del problema di clustering e la ram mangiata voglio dire dello stile applicato e il click dei marker

```
1  export const buildLayer = (mapLayer: MapLayer) => {
2
3      if (!mapLayer.layer) {
4          return undefined;
5      }
6
7      switch (mapLayer.layer.type) {
8          //...
9          case 'tile': {
10             const layer = new TileLayer({
11                 source: new TileImage({
12                     url: mapLayer.layer.url,
13                     crossOrigin: mapLayer.layer.crossOrigin,
14                 }),
15                 opacity: mapLayer.opacity / 100,
16                 visible: mapLayer.visible,
```

```
17                  zIndex: mapLayer.zIndex,
18              });
19              layer.set('id', mapLayer.key);
20              return layer;
21          }
22          case 'geojson': {
23              const source = new VectorSource({
24                  format: new GeoJSON({
25                      dataProjection: mapLayer.projection?.name,
26                      featureProjection: mapLayer.projection?.
    name,
27                  }),
28                  url: mapLayer.layer.url,
29              });
30
31              layer.set('id', mapLayer.key);
32              return layer;
33          }
34          //...
35      }
36  }
```

Listing 5.5: Implementazione BuildLayer

Bibliografia

[1] Componenti base di OpenLayers. https://openlayers.org/doc/tutorials/concepts.html.

[2] Esempio di integrazione del protocollo WMS con OpenLayers.

[3] GeoServer Documentazione. https://docs.geoserver.org/stable/en/user/rest/.

[4] GeoServer Github. https://github.com/geoserver/geoserver.

[5] OpenLayer Documentation. https://openlayers.org/en/latest/apidoc/.

[6] OpenLayers Github. https://github.com/openlayers/openlayers.

[7] Tile Map Service Specification. https://wiki.osgeo.org/wiki/Tile_Map_Service_Specification, 2012-05-16.

[8] Angular. Documentazione di Angular. https://angular.dev/tutorials/first-app.

[9] Jeff de la Beaujardiere. OpenGIS Web Map Service (WMS) Implementation Specification. https://portal.ogc.org/files/?artifact_id=14416, 2006-03-15.

[10] Ministero dell'Ambiente e della Sicurezza Energetica. Mappe fornite mediante protocolli OGC dal GeoPortale Nazionale. https://gn.mase.gov.it/portale/servizi-di-rete-ogc.

[11] GeoCart. Dati geospaziali forniti da GeoCart. https://www.geocartspa.it/.

[12] Keith Pomakis Joan Masó and Núria Julià. OpenGIS Web Map Tile Service Implementation Standard. https://portal.ogc.org/files/?artifact_id=35326, 2010-04-06.

[13] Wikipedia. Open Geospatial Consortium. https://en.wikipedia.org/wiki/Open_Geospatial_Consortium, 2023-17-12.